

IPC144 – Introduction to C Programming

Practice Exercises

Week 13 – Data Structures

12.6 (Concatenating Lists) Write a program that concatenates two linked lists of characters. The program should include function `concatenate` that takes pointers to both lists as arguments and concatenates the second list to the first list.

12.7 (Merging Ordered Lists) Write a program that merges two ordered lists of integers into a single ordered list of integers. Function `merge` should receive pointers to the first node of each of the lists to be merged and return a pointer to the first node of the merged list.

12.8 (Inserting into an Ordered List) Write a program that inserts 25 random integers from 0 to 100 in order in a linked list. The program should calculate the sum of the elements and the floating point average of the elements.

12.9 (Creating a Linked List, Then Reversing Its Elements) Write a program that creates a linked list of 10 characters, then creates a copy of the list in reverse order.

12.10 (Reversing the Words of a Sentence) Write a program that inputs a line of text and uses a stack to print the line reversed.

12.11 (Palindrome Tester) Write a program that uses a stack to determine whether a string is a palindrome (i.e., the string is spelled identically backward and forward). The program should ignore spaces and punctuation.

12.12 (Infix-to-Postfix Converter) Stacks are used by compilers to help in the process of evaluating expressions and generating machine-language code. In this and the next exercise, we investigate how compilers evaluate arithmetic expressions consisting only of constants, operators and parentheses.

Humans generally write expressions like $3 + 4$ and $7 / 9$ in which the operator (+ or / here) is written between its operands—this is called **infix notation**. Computers “prefer” **postfix notation** in which the operator is written to the right of its two operands. The preceding infix expressions would appear in postfix notation as $3\ 4\ +$ and $7\ 9\ /$, respectively.

To evaluate a complex infix expression, some compilers first convert the expression to postfix notation, and then evaluate the postfix version. Each of these algorithms requires only a single left to-right pass of the expression. Each algorithm uses a stack in support of its operation, and in each the stack is used for a different purpose.

In this exercise, you'll write a version of the infix-to-postfix conversion algorithm. In the next exercise, you'll write a version of the postfix-expression evaluation algorithm.

Write a program that converts an ordinary infix arithmetic expression (assume a valid expression is entered) with single-digit integers such as

$(6 + 2) * 5 - 8 / 4$

to a postfix expression. The postfix version of the preceding infix expression is

$6 2 + 5 * 8 4 / -$

The program should read the expression into character array `infix` and use the stack functions implemented in this chapter to help create the postfix expression in character array `postfix`. The algorithm for creating a postfix expression is as follows:

1) Push a left parenthesis '(' onto the stack.

2) Append a right parenthesis ')' to the end of `infix`.

3) While the stack is not empty, read `infix` from left to right and do the following:

If the current character in `infix` is a digit, copy it to the next element of `postfix`.

If the current character in `infix` is a left parenthesis, push it onto the stack.

If the current character in `infix` is an operator,

Pop operators (if there are any) at the top of the stack while they have equal or higher precedence than the current operator, and insert the popped operators in `postfix`.

Push the current character in `infix` onto the stack.

If the current character in `infix` is a right parenthesis

Pop operators from the top of the stack and insert them in `postfix` until a left parenthesis is at the top of the stack.

Pop (and discard) the left parenthesis from the stack.

The following arithmetic operations are allowed in an expression:

+ addition

- subtraction

* multiplication

/ division

^ exponentiation

% remainder

The stack should be maintained with the following declarations:

```
struct stackNode {  
    char data;  
    struct stackNode *nextPtr;  
};  
typedef struct stackNode StackNode;  
typedef StackNode *StackNodePtr;
```

The program should consist of main and eight other functions with the following function headers:

```
void convertToPostfix(char infix[], char postfix[])
```

Convert the infix expression to postfix notation.

```
int isOperator(char c)
```

Determine whether c is an operator.

```
int precedence(char operator1, char operator2)
```

Determine whether the precedence of operator1 is less than, equal to, or greater than the precedence of operator2. The function returns -1, 0 and 1, respectively.

```
void push(StackNodePtr *topPtr, char value)
```

Push a value on the stack.

```
char pop(StackNodePtr *topPtr)
```

Pop a value off the stack.

```
char stackTop(StackNodePtr topPtr)
```

Return the top value of the stack without popping the stack.

```
int isEmpty(StackNodePtr topPtr)
```

Determine whether the stack is empty.

```
void printStack(StackNodePtr topPtr)
```

Print the stack.

12.13 (Postfix Evaluator) Write a program that evaluates a postfix expression (assume it's valid)

such as

6 2 + 5 * 8 4 / -

The program should read a postfix expression consisting of single digits and operators into a character array. Using the stack functions implemented earlier in this chapter, the program should scan the expression and evaluate it. The algorithm is as follows:

1) Append the null character ('\0') to the end of the postfix expression. When the null character is encountered, no further processing is necessary.

2) While '\0' has not been encountered, read the expression from left to right.

If the current character is a digit,

Push its integer value onto the stack (the integer value of a digit character is its value in the computer's character set minus the value of '0' in the computer's character set).

Otherwise, if the current character is an *operator*,

Pop the two top elements of the stack into variables x and y.

Calculate $y \text{ operator } x$.

Push the result of the calculation onto the stack.

3) When the null character is encountered in the expression, pop the top value of the stack.

This is the result of the postfix expression.

[Note: In 2) above, if the operator is '/', the top of the stack is 2, and the next element in the stack is 8, then pop 2 into x, pop 8 into y, evaluate $8 / 2$, and push the result, 4, back onto the stack. This note also applies to the other binary operators.]

The arithmetic operations allowed in an expression are:

+ addition

- subtraction

* multiplication

/ division

^ exponentiation

% remainder

The stack should be maintained with the following declarations:

```
struct stackNode {  
    int data;  
    struct stackNode *nextPtr;  
};  
  
typedef struct stackNode StackNode;  
  
typedef StackNode *StackNodePtr;
```

The program should consist of main and six other functions with the following function headers:

```
int evaluatePostfixExpression(char *expr)
```

Evaluate the postfix expression.

```
int calculate(int op1, int op2, char operator)
```

Evaluate the expression op1 operator op2.

```
void push(StackNodePtr *topPtr, int value)
```

Push a value on the stack.

```
int pop(StackNodePtr *topPtr)
```

Pop a value off the stack.

```
int isEmpty(StackNodePtr topPtr)
```

Determine whether the stack is empty.

```
void printStack(StackNodePtr topPtr)
```

Print the stack.

12.14 (Postfix Evaluator Modification) Modify the postfix evaluator program of Exercise 12.13

so that it can process integer operands larger than 9.

12.15 (Supermarket Simulation) Write a program that simulates a check-out line at a supermarket. The line is a queue. Customers arrive in random integer intervals of 1 to 4 minutes. Also, each customer is serviced in random integer intervals of 1 to 4 minutes. Obviously, the rates need to be balanced. If the average arrival rate is larger than the average service rate, the queue will grow infinitely.

Even with balanced rates, randomness can still cause long lines. Run the supermarket simulation for a 12-hour day (720 minutes) using the following algorithm:

1) Choose a random integer between 1 and 4 to determine the minute at which the first customer arrives.

2) At the first customer's arrival time:

Determine customer's service time (random integer from 1 to 4);

Begin servicing the customer;

Schedule arrival time of next customer (random integer 1 to 4 added to the current time).

3) For each minute of the day:

If the next customer arrives,

Say so;

Enqueue the customer;

Schedule the arrival time of the next customer.

If service was completed for the last customer,

Say so;

Dequeue next customer to be serviced;

Determine customer's service completion time

(random integer from 1 to 4 added to the current time).

Now run your simulation for 720 minutes and answer each of the following:

a) What's the maximum number of customers in the queue at any time?

b) What's the longest wait any one customer experienced?

c) What happens if the arrival interval is changed from 1 to 4 minutes to 1 to 3 minutes?

12.16 (*Allowing Duplicates in a Binary Tree*) Modify the program of Fig. 12.19 to allow the binary tree to contain duplicate values.

12.17 (*Binary Search Tree of Strings*) Write a program based on the program of Fig. 12.19 that inputs a line of text, tokenizes the sentence into separate words, inserts the words in a binary search tree, and prints the in-order, pre-order, and post-order traversals of the tree.

[*Hint:* Read the line of text into an array. Use strtok to tokenize the text. When a token is

found, create a new node for the tree, assign the pointer returned by strtok to member string of the new node, and insert the node in the tree.]

12.18 (Duplicate Elimination) We've seen that duplicate elimination is straightforward when creating a binary search tree. Describe how you would perform duplicate elimination using only a one dimensional array. Compare the performance of array-based duplicate elimination with the performance of binary-search-tree-based duplicate elimination.

12.19 (Depth of a Binary Tree) Write a function depth that receives a binary tree and determines how many levels it has.

12.20 (Recursively Print a List Backward) Write a function printListBackward that recursively outputs the items in a list in reverse order. Use your function in a test program that creates a sorted list of integers and prints the list in reverse order.

12.21 (Recursively Search a List) Write a function searchList that recursively searches a linked list for a specified value. The function should return a pointer to the value if it's found; otherwise, NULL should be returned. Use your function in a test program that creates a list of integers. The program should prompt the user for a value to locate in the list.

12.22 (Binary Tree Search) Write function binaryTreeSearch that attempts to locate a specified value in a binary search tree. The function should take as arguments a pointer to the root node of the binary tree and a search key to be located. If the node containing the search key is found, the function should return a pointer to that node; otherwise, the function should return a NULL pointer.

12.23 (Level Order Binary Tree Traversal) The program of Fig. 12.19 illustrated three recursive methods of traversing a binary tree—inorder traversal, preorder traversal, and postorder traversal.

This exercise presents the **level order traversal** of a binary tree in which the node values are printed

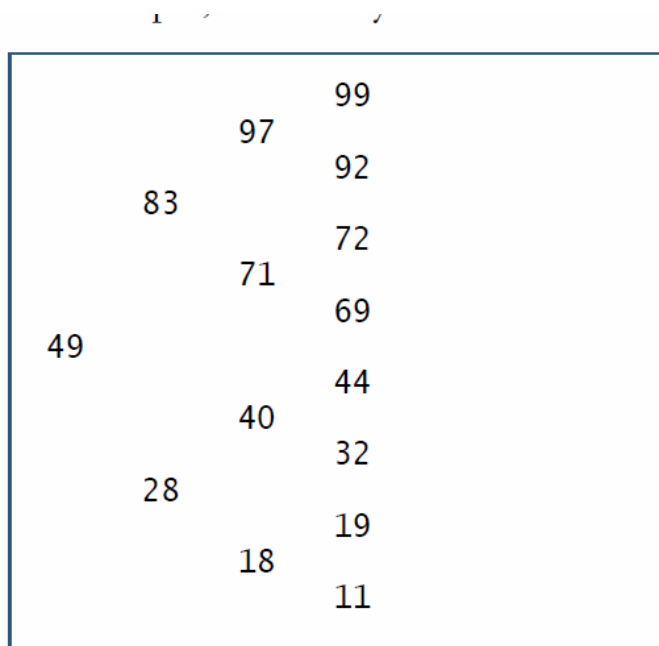
level-by-level starting at the root node level. The nodes on each level are printed from left to right.

The level order traversal is not a recursive algorithm. It uses the queue data structure to control the output of the nodes. The algorithm is as follows:

- 1) Insert the root node in the queue
- 2) While there are nodes left in the queue,
 - Get the next node in the queue
 - Print the node's value
 - If the pointer to the left child of the node is not NULL
 - Insert the left child node in the queue
 - If the pointer to the right child of the node is not NULL
 - Insert the right child node in the queue.

Write function `levelOrder` to perform a level order traversal of a binary tree. The function should take as an argument a pointer to the root node of the binary tree. Modify the program of Fig. 12.19 to use this function. Compare the output from this function to the outputs of the other traversal algorithms to see that it worked correctly. [Note: You'll also need to modify and incorporate the queue-processing functions of Fig. 12.13 in this program.]

12.24 (Printing Trees) Write a recursive function `outputTree` to display a binary tree on the screen. The function should output the tree row-by-row with the top of the tree at the left of the screen and the bottom of the tree toward the right of the screen. Each row is output vertically. For example, the binary tree illustrated in Fig. 12.22 is output as follows:



Note that the rightmost leaf node appears at the top of the output in the rightmost column, and the root node appears at the left of the output. Each column of output starts five spaces to the right of the previous column. Function `outputTree` should receive as arguments a pointer to the root node of the tree and an integer `totalSpaces` representing the number of spaces preceding the value to be output (this variable should start at zero so that the root node is output at the left of the screen). The function uses a modified inorder traversal to output the tree. The algorithm is as follows:

While the pointer to the current node is not NULL

 Recursively call `outputTree` with the current node's right subtree and `totalSpaces + 5`.

 Use a for statement to count from 1 to `totalSpaces` and output spaces.

 Output the value in the current node.

 Recursively call `outputTree` with the current node's left subtree and `totalSpaces + 5`.